

PortionSpot POS

What each system can do on its own, and what only works when they are joined. A capability map and device-test reference — 16 August 2026.

Two clients read and write one Supabase database. This document says what each is **designed** to do alone, what it is designed to do connected, and where the boundaries actually sit — so a device test can tell a missing feature from a broken one.

How to read this

Every capability listed was read out of the source on 16 August 2026, or queried against the live database — not recalled. What is **not** established is behaviour on real hardware: the money release described in section 5 is compile-green and unit-green on both sides (510 Android tests, 662 web tests) and **has never run on a phone or against a real day's trading**. Treat section 5 as the thing to attack first.

Where everything lives

System	What it is	Open it
Web back office	React + Dexie (IndexedDB) PWA, deployed on Vercel	portionspot-pos-v1.vercel.app source · this release · PR #6
Android till	Kotlin + Jetpack Compose + Room, installed by APK	source branch sync-repoint-web-schema · this release
The database	Supabase Postgres. Both clients speak to pos2_* views over the same base tables.	production project throwaway test project

The two Supabase projects are identical in shape. Test destructive things against the throwaway one; it exists so a bad sync cannot cost a real day's takings.

1 · The two systems side by side

They are not the same app in two skins. Each owns something the other deliberately does not.

	Android till	Web back office
Built for	Standing at the counter, one phone per cashier	Sitting down with the whole shop's numbers
Signing in	Username + PIN checked against the roster already on the phone. No tokens, no network needed.	Username + PIN in the browser. Same PBKDF2 hash format.
Works with no internet	Yes — and it boots into local mode by default. Cloud is opt-in.	Yes, once loaded. It is a PWA and all data is in IndexedDB.
Owens the catalogue	No. Items are pull-only : the till reads products, never writes them.	Yes. Adding, pricing and costing products happens here.
Reads payment SMS	Yes. It can read the EcoCash / OneMoney / InnBucks / Omari confirmation from the inbox itself.	No — a browser cannot touch an inbox. It has a paste-in box instead.
Prints	Yes, to a thermal printer over RawBT, plus PDF.	Browser print and PDF export.
Screens	15 (4 pinned, 11 in the overflow) plus a separate 6-tab admin shell	20 routes across a Till area and a Back-office area

2 · The Android till on its own, with no cloud

This is the default state on a fresh install. A shop with one phone, no Supabase account and no internet should be able to trade all day. Everything below is device-local and complete without a network.

Should work

Screen	What you should be able to do
Point of Sale	Ring up a sale; search or scan; sell by piece, by box or by measured quantity; per-item discount and per-item cashier markup; whole-sale discount behind a PIN gate; VAT on or off; split the payment across tenders; take a note and give change; park a sale and come back to it; raise a quote that is not yet a sale.
Inventory	Add and edit products, cost prices, stock, categories, low-stock thresholds, product photos, and car-fitment tags. Adjust stock with a reason.
Customers	Create a customer, set a credit limit, sell on account, take a repayment, see who owes what and how old the debt is.
Change & Credit	The two directions of owing, kept apart: what customers owe the shop, and what the shop owes back as change it could not give or a refund not yet paid.
Receipts / Refunds	Find a past sale, reprint it, refund all or part of it, choose whether the goods go back on the shelf, and pay the refund out in any tender.
Till & Safe	Open a shift with a float, record pay-ins, pay-outs, petty cash, drops to the safe and float top-ups, count the drawer, and close the day.
Expenses / Suppliers / POs	Record what the shop spent, keep a supplier list, raise a purchase order and receive it into stock.
Dashboard / Sales	The money figures in section 5, over any window.
Mobile Money	Read a payment confirmation SMS off the phone and match it to a sale or a debt.
Admin shell	Admins land in a six-tab shell: Dashboard, Reports, Inventory, Alerts, Admin, Settings, with a way down into the cashier POS.
Settings	Shop details, VAT, currency and rate, receipt content, theme, printer, and a device PIN.

Cannot work without the cloud

Thing	Why
A second phone seeing the first phone's sales	There is nothing between them. Every till is its own island until sync is on.
Cashier accounts created elsewhere	The roster is whatever has synced to this device. On a fresh local install that is the local admin.
Anything the web owns	Notably the product catalogue: the till reads items, it does not publish them.

3 · The web back office on its own

Open [the app](#) and it works against IndexedDB in that browser, on that device, with no Supabase connection at all. It is a full POS in its own right, not a viewer.

Area	Routes	What you should be able to do
Till	Sell, Receipts, Refunds, Customers, Credit, Payments, Cash up	Everything the Android till does at the counter, in a browser: sell, refund, take repayments, record a pasted payment SMS, and count the drawer at the end of a shift.

Area	Routes	What you should be able to do
Back office	Dashboard, Reports, Inventory, Purchase orders, Suppliers, Expenses, Cash book, Staff, Alerts, Audit log, Sync, Settings	Own the catalogue and its cost prices; run the day close; move money between till and safe; manage the staff roster and their permissions; read the audit trail; export CSV and PDF.

- **The Cash book is the fullest money screen either system has.** Its day close prints a column that adds up line by line, and it is the one to check first after this release.
- **Two things never leave the browser**, by design: device settings and the alert feed. A notification raised here stays here.
- **It is the catalogue's home.** If a product is wrong on the phone, it is fixed here.

4 · The two of them joined to one database

Point both at the same Supabase project and 24 tables move between them. Sync is a pull then a push, cursor-based per table, and safe to interrupt: a pull that dies half way resumes where it stopped rather than re-reading everything.

What flows, and which way

Direction	Tables
Web to phone only	items — the product catalogue. The till never writes it. Stock levels are not written either: <code>stock_movements</code> is the authority and each device recomputes the on-hand figure after a pull.
Both ways	<code>item_attributes</code> · <code>customers</code> · <code>sales</code> · <code>sale_items</code> · <code>sale_payments</code> · <code>refunds</code> · <code>refund_items</code> · <code>refund_payments</code> · <code>credit_txns</code> · <code>stock_movements</code> · <code>mobile_money_receipts</code> · <code>cash_sessions</code> · <code>cash_movements</code> · <code>expenses</code> · <code>suppliers</code> · <code>purchase_orders</code> · <code>purchase_order_items</code> · <code>audit_entries</code> · <code>staff</code>
Never leaves the device	Both: settings, notifications. Android also: <code>staff_requests</code> (the admin–cashier approval channel), <code>day_closes</code> (the detailed till/safe record), <code>outside_funds</code> (owner money vs a loan). Each is a decision, not an oversight — there is no cloud column that expresses them.

What that buys you

- **One shop, many phones.** A sale rung up on any device appears on every other device and in the web back office, with its lines, its tenders and its stock movement intact.
- **One drawer.** The database allows exactly one open cash session per shop — not per till. Every device is counting the same physical drawer, and closing it closes it for everyone. Two tills that each opened a shift while offline are merged onto one on sync.
- **A refund traced to its sale.** Refunds are first-class rows with a link back, not negative-total sales.
- **Permissions that can be taken away.** Revoking a capability on the roster reaches the cashier's phone on its next sync.
- **The owner's phone sees the shop.** Till shortages, voids and receipt edits recorded on a cashier's phone land in the shared audit trail.

The one that will bite you if you forget it

Alerts do not cross devices. The notification feed is local on both clients. A low-stock warning or a large-sale alert raised on a cashier's phone stays on that phone — it is not a sync failure, there is no shared table for it yet. Same for the Android day-close detail and the owner-money-vs-loan split.

5 · The money model — new on both sides, untested on either

Until this release the two clients could give different answers to "what did I make today". They now run the same algorithm: [cashBasis.js](#) on the web is a port of [CashBasis.kt](#) on Android, and ten numbered test vectors are asserted on both.

What you should now see on screen

Rule	What it looks like in the shop
Money counts when it arrives, not when the receipt is written	Sell \$100 on account and the day's revenue does not move . The sale, the stock movement and the debt are all recorded. Take the \$100 next week and it counts next week .
Cost is pro-rated to what was collected	A \$100 sale costing \$60 with \$40 paid shows revenue 40, cost 24, profit 16. It never reads as a loss.
Change owed is not revenue	Hand over \$100 for a \$90 sale and only \$90 is recognised. The \$10 you could not give back is a liability.
A refund reverses on the day the goods came back	Yesterday's figures do not change. That day may already have a counted drawer against it.
Refunding an unpaid credit sale reverses nothing	Nothing had been recognised. What gets cancelled is the debt.
VAT is signed	In a window holding only a refund the VAT line reads returned , not collected, and the card still adds up.
Two sums that must always hold	$\text{collected} - \text{refunded} - \text{VAT} = \text{revenue and costed}$ $\text{revenue} - \text{cost of goods} = \text{profit}$. Both are printed in words under the figures so you can check them by eye.
"Left unpaid at the till" is not "still owed"	It is frozen at what walked out unpaid that day, and settling the debt later does not reduce it. What is genuinely outstanding is in Change & Credit.

Worth attacking on a real device

- Sell on credit, then part-pay, then refund part of it, on three different days — and check each day separately.
- Split a tender across cash and EcoCash and confirm the drawer moves by the cash half only.
- Close a day, then refund a sale from it, and confirm the closed day's figures are untouched.
- Run the same window on the phone and in the web back office and confirm they agree to the cent.

6 · Known limits, stated plainly

Limit	Detail
Not device-tested	The money release is unit-green on both sides and has never run on hardware or against a real day.
Alerts are per-device	No shared notification table exists yet.
Android does not yet write the tender on a debt repayment	The column and the web payload have been ready since 9 August. Until the till fills it in, a repayment synced from a phone reads as unknown tender and is treated as drawer-neutral — deliberately, because guessing would make a cash-up less true, not more.
Two known Android money bugs still open	A mobile-money debt payment skips the overpayment-to-change-owed split, and a customer payout assumes cash.
Billed-basis reports still filter on one sale status	Fifteen queries on the phone were left alone on purpose in this pass; they are not the pair that was inconsistent, and rewriting them blind is how a fix becomes an incident.

7 · Everything worth clicking

Live	Web POS (production)
Code	Web repository · Android repository · Android release branch
This release	Web: PR #6 · web commit on main · Android commit
Database	Supabase production · Supabase throwaway (test here)
The money model	cashBasis.js (web) · CashBasis.kt (Android) · the ten test vectors
What syncs	SyncConfig.kt — the table list and the local-only list · sync.is (web)
The write-ups	Web reply to the Android brief · Android record of the three fixes

Compiled 16 August 2026 from the source of both systems. Where this document and the code disagree, the code is right and this document is a bug.